实验四报告

姓名：李祥熙　学号：2234412799

2026 年 4 月 22 日

1　实验目的

本实验围绕无监督聚类展开，目标如下：

- 在二维点数据、时序数据和 UCI 数据集上完成聚类实验，并比较不同算法的适应性。
- 手工实现 KMeans 与 single-link 层次聚类（SLINK），并与 sklearn 实现进行对照。
- 通过聚类图、ARI、NMI、运行时间和内存占用分析参数对结果的影响。

2　实验过程

2.1　实验环境

- 操作系统：Linux
- 编程语言：Python 3
- 主要工具：NumPy、Pandas、Matplotlib、scikit-learn
- 报告目录结构参考了 `Machine-Learning/lab-cnn-rnn/lab-report`

2.2　数据集与预处理

- 二维点数据：`dots/clusterData*.dat`，每行包含一个二维坐标点，主要用于观察不同聚类算法对几何形状的划分效果。
- 时序数据：`timeSeries/Plane_TRAIN.txt`，首列为标签，其余为时序值。实验中对数值部分做标准化，并进一步提取统计特征。
- UCI 数据：`wine+quality/winequality-red.csv` 与 `winequality-white.csv`，将两份数据合并后以质量标签作为参考类别。

2.3 在原有文件基础上的改动

原始工程中，`kmeans.py`、`linkage.py` 和 `dbscan.py` 主要是对 sklearn 的简单封装，`clustering.py` 负责读入数据并调用模型。为了满足实验要求，在原结构上做了以下改动：

- 新增 `features.py`，为点数据和时序数据提供统一的特征处理入口。
- 将 `kmeans.py` 改为支持手写 KMeans 与 sklearn 双版本对照。
- 将 `linkage.py` 改为支持手写 SLINK（single-link）与 sklearn 双版本对照。
- 扩展 `clustering.py`，加入 UCI 数据读取、结果评估、时间/内存统计、参数扫描和统一输出目录 `results/`。

2.4 实现步骤

1. 读取不同类型的数据集并完成归一化或特征提取。
2. 调用 KMeans、single-link 层次聚类和 DBSCAN 进行聚类，并支持手写实现与 sklearn 实现对照。
3. 使用 ARI、NMI、运行时间、峰值内存和聚类散点图对结果进行评估与分析。

3 实验内容

3.1 任务一：KMeans 与 single-link 层次聚类对照

KMeans 通过迭代更新簇中心和样本归属来最小化簇内平方误差，适合紧凑、近似球形的数据簇。single-link 层次聚类则按照最近邻距离逐步合并簇，更容易形成链式结构。

主要参数：

- KMeans 的簇数设为 5，初始化方式为 random。
- 层次聚类使用 single linkage，并同样设置簇数为 5。
- 为了比较实现差异，手写版和 sklearn 版都执行一次。

实现要点：

- KMeans 手写版直接使用 NumPy 计算样本到中心的距离、更新中心并判断收敛。
- SLINK 手写版通过最近邻链接关系构造单链层次结构，再切分出指定簇数。

3.2 任务二：DBSCAN 与参数敏感性

DBSCAN 是基于密度的聚类方法，不需要预先指定簇数，可以检测任意形状簇并将低密度区域判为噪声。其核心参数是 `eps` 和 `min_samples`，对尺度非常敏感。

3.3 任务三：不同数据集上的聚类表现

在时序数据和 UCI 数据集上，使用 ARI 和 NMI 衡量聚类标签与真实标签的一致性；在二维点数据上主要观察聚类图的几何划分效果。

4 实验结果

4.1 任务一结果：二维点数据的聚类图

二维点数据对应的图像如下，结果都保存到 `results/` 下，并在图例中标明算法、实现方式和参数。

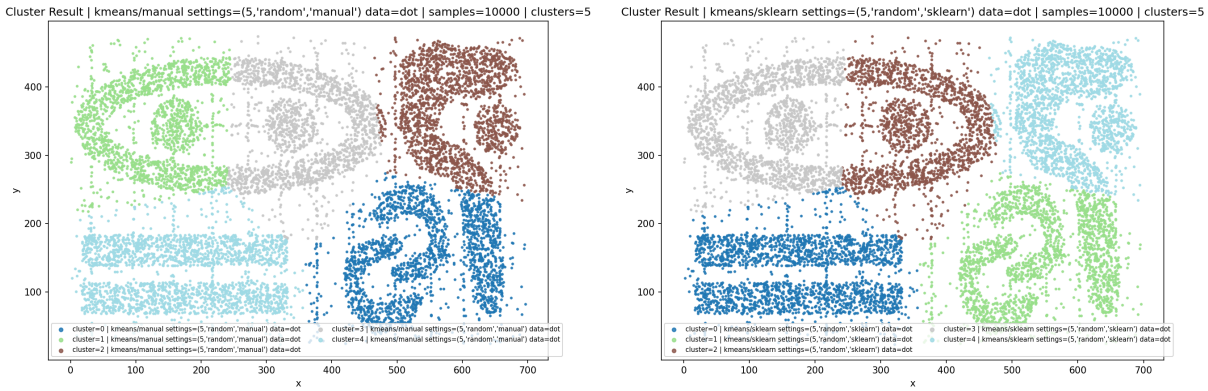

图 1: 二维点数据上的 KMeans 结果对比（左：手写版，右：sklearn 版）

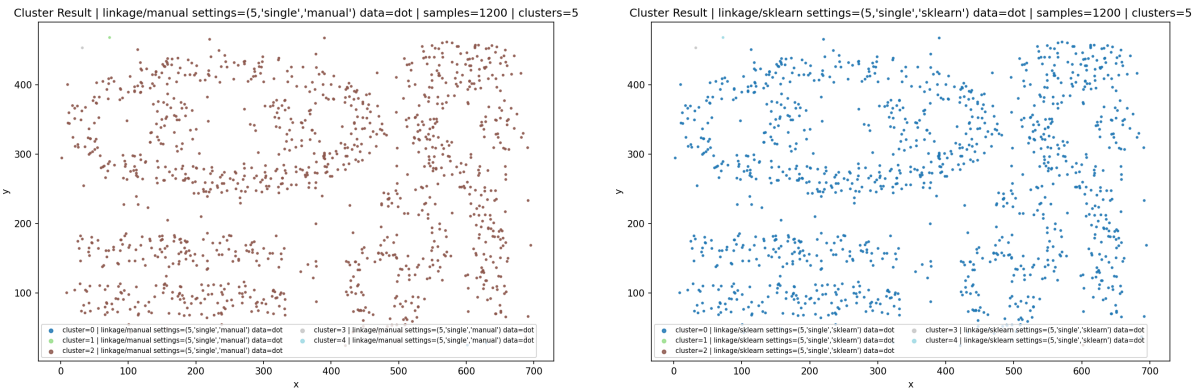

图 2: 二维点数据上的层次聚类结果对比（左：手写版，右：sklearn 版）

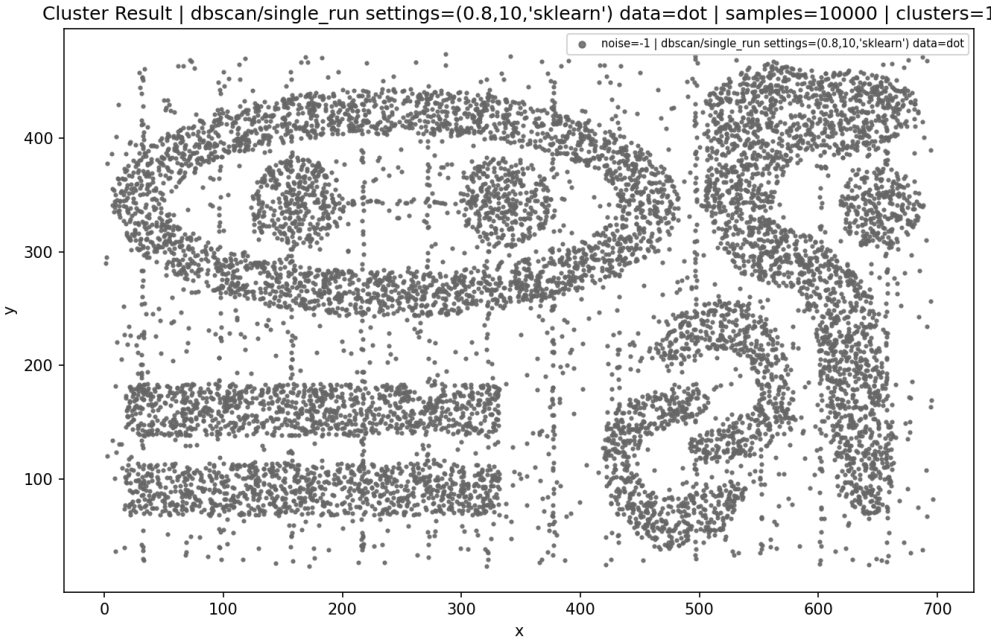

图 3: 二维点数据上的 DBSCAN 结果

结果分析:

- KMeans 的手写版和 sklearn 版图形几乎一致，说明手写实现与标准实现基本等价。
- KMeans 对这类弯曲、细长结构的拟合能力有限，因此能分出若干区域，但不能完整恢复所有非凸形状。
- single-link 层次聚类容易发生“链式合并”，一些相邻但语义上不同的区域会被连成一个大簇。
- DBSCAN 的这组参数偏严格，图中噪声点较多，说明 `eps` 对当前数据尺度过小，参数需要重新调整。

4.2 任务二结果：时序数据聚类指标

时序数据的实验结果如下。

表 1: 时序数据聚类结果

算法	实现	ARI	NMI	运行时间 (s)
KMeans	manual	0.3360	0.5117	0.0009
KMeans	sklearn	0.3341	0.5104	0.0550
linkage	manual	0.0019	0.0870	0.0194
linkage	sklearn	0.0019	0.0870	0.0225
DBSCAN	sklearn	0.0032	0.0408	0.0018

结果分析：

- KMeans 在该数据上效果最好，ARI 和 NMI 都明显高于另外两种方法，说明特征空间中存在一定的中心型结构。
- 手写 KMeans 与 sklearn KMeans 的指标几乎一致，说明实现正确。
- 层次聚类和 DBSCAN 的效果较弱，说明这些数据在当前特征表示下并不适合直接用单链合并或密度阈值划分。

4.3 任务三结果：UCI 数据聚类指标

UCI wine 数据上的结果如下。

表 2: UCI wine 数据聚类结果

算法	实现	ARI	NMI	运行时间 (s)
KMeans	manual	0.0342	0.0564	0.0451
KMeans	sklearn	0.0325	0.0536	0.2207
linkage	manual	0.0001	0.0099	2.4243
linkage	sklearn	0.0001	0.0099	0.0325
DBSCAN	sklearn	-0.0010	0.0466	0.1295

结果分析：

- 三种算法在 wine 数据上的指标整体较低，说明原始质量标签与无监督聚类结构并不完全一致。
- KMeans 虽然比另外两种方法稍好，但提升幅度有限，表明该数据集更适合监督学习或更有针对性的特征工程。
- 手写层次聚类在 1200 条采样数据上耗时明显更高，说明单链层次聚类对样本规模较敏感。

5 总结

本实验完成了聚类算法的实现、对照与分析：

- 在原有工程基础上补充了手写 KMeans、手写 SLINK、特征处理、统一结果输出和评估流程。
- 在二维点、时序数据和 UCI 数据上比较了 KMeans、层次聚类与 DBSCAN 的适应性。

- 结果表明：KMeans 更适合紧凑型簇，single-link 易链式合并，DBSCAN 对参数和尺度最敏感。

本次实验也说明，聚类算法的表现不仅取决于算法本身，还强烈依赖于数据表示、参数选择和评价方式。对于复杂数据，先做合适的特征工程和参数搜索往往比直接套用算法更重要。

附录：实验源代码与运行命令

A.1 clustering.py

```
    # -*- coding: utf-8 -*-
import argparse
import ast
import os
import time
import tracemalloc

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn.metrics as skm
import sklearn.preprocessing as skp

import dbscan as db
import features as ft
import kmeans as km
import linkage as lk


RESULT_DIR = "results"


def ensure_result_dir():
    os.makedirs(RESULT_DIR, exist_ok=True)


def load_time_series(path, separator=","):
    print("开始读入时序数据...")
    start_t = time.time()
    df = pd.read_table(path, header=None, sep=separator)
```

```
    data = df.to_numpy(dtype=np.float64)
    x = data[:, 1:]
    y = data[:, 0]
    x = skp.StandardScaler().fit_transform(x)
    end_t = time.time()
    print("时序数据读入结束。耗时%.3f秒。" % (end_t - start_t))
    return x, y


def load_dots(path, separator=r"\s+"):
    print("开始读入二维点数据...")
    start_t = time.time()
    df = pd.read_table(path, header=None, sep=separator)
    data = df.to_numpy(dtype=np.float64)
    end_t = time.time()
    print("二维点数据读入结束。耗时%.3f秒。" % (end_t - start_t))
    return data


def load_uci_wine(path):
    print("开始读入UCI wine质量数据...")
    start_t = time.time()

    if os.path.isdir(path):
        red_path = os.path.join(path, "winequality-red.csv")
        white_path = os.path.join(path, "winequality-white.csv")
        red = pd.read_csv(red_path, sep=";")
        white = pd.read_csv(white_path, sep=";")
        data = pd.concat([red, white], axis=0, ignore_index=True)
    else:
        data = pd.read_csv(path, sep=";")

    x = data.iloc[:, :-1].to_numpy(dtype=np.float64)
    y = data.iloc[:, -1].to_numpy()
    x = skp.StandardScaler().fit_transform(x)

    end_t = time.time()
    print("UCI数据读入结束。耗时%.3f秒。" % (end_t - start_t))
    return x, y
```

```python
def load_data(path, data_type):
    x_train = y_train = None
    if data_type == "dot":
        x_train = load_dots(path)
        x_train, _meta = ft.dotFeature(x_train, None)
    elif data_type == "timeseries":
        x_ts, y_train = load_time_series(path)
        x_train, _meta = ft.timeSeriesFeature(x_ts, None)
    elif data_type == "uci":
        x_train, y_train = load_uci_wine(path)
    else:
        raise ValueError("%s是未知数据类型。" % data_type)
    return x_train, y_train


def nmi(y_true, y_pred):
    y_true = np.squeeze(y_true)
    return skm.normalized_mutual_info_score(y_true, y_pred)


def ari(y_true, y_pred):
    y_true = np.squeeze(y_true)
    return skm.adjusted_rand_score(y_true, y_pred)


def show_dots(x, labels, output_path, plot_title, method_desc):
    unique_labels = np.unique(labels)
    fig, ax = plt.subplots(1, figsize=(9, 6))
    cmap = plt.cm.get_cmap("tab20", max(1, unique_labels.size))
    for idx, lb in enumerate(unique_labels):
        color = "#666666" if lb == -1 else cmap(idx)
        cluster_tag = "noise" if lb == -1 else "cluster"
        label_text = "%s=%s | %s" % (cluster_tag, lb, method_desc)
        ax.scatter(x[labels == lb, 0], x[labels == lb, 1], s=4, c=[
            color], label=label_text, alpha=0.85)
    ax.set_title(plot_title)
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.legend(markerscale=2, fontsize=7, ncol=2, loc="best")
    plt.tight_layout()
    plt.savefig(output_path, dpi=150)
```

```
    plt.close(fig)


def run_with_profile(train_func, x, settings):
    tracemalloc.start()
    start_t = time.perf_counter()
    cluster = train_func(x, *settings)
    elapsed = time.perf_counter() - start_t
    _curr, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return cluster, elapsed, peak / (1024 * 1024)


def evaluate_result(x, y, cluster, model_name, run_name, data_type,
   data_path, settings):
    print("-" * 70)
    print("%s [%s]" % (model_name, run_name))
    if y is None or len(y) == 0:
        ensure_result_dir()
        data_name = os.path.splitext(os.path.basename(data_path))[0]
        safe_settings = str(settings).replace(" ", "").replace("\"", ""
           )
        output_name = "%s_%s_%s_%s.png" % (model_name, run_name,
           data_name, data_type)
        output_path = os.path.join(RESULT_DIR, output_name)
        method_desc = "%s/%s settings=%s data=%s" % (model_name,
           run_name, safe_settings, data_type)
        plot_title = "Cluster Result | %s | samples=%d | clusters=%d" %
            (
            method_desc,
            x.shape[0],
            np.unique(cluster.labels_).size,
        )
        show_dots(x, cluster.labels_, output_path, plot_title,
           method_desc)
        print("无标签数据。聚类可视化已保存到: %s" % output_path)
    else:
        pred_label = cluster.labels_
        print("ARI=%.4f, NMI=%.4f" % (ari(y, pred_label), nmi(y,
           pred_label)))
```

```
def run_model(model_name, x, y, settings, compare, data_type, data_path
    ):
    if model_name == "kmeans":
        train_func = km.train
        base_settings = settings if settings is not None else (5, "
            random")
    elif model_name == "linkage":
        train_func = lk.train
        base_settings = settings if settings is not None else (5, "
            single")
    elif model_name == "dbscan":
        train_func = db.train
        base_settings = settings if settings is not None else (0.8, 10)
    else:
        raise ValueError("未知模型: %s" % model_name)

    results = []
    if model_name in ("kmeans", "linkage") and compare:
        manual_settings = tuple(base_settings) + ("manual",)
        skl_settings = tuple(base_settings) + ("sklearn",)

        manual_cluster, manual_time, manual_mem = run_with_profile(
            train_func, x, manual_settings)
        evaluate_result(x, y, manual_cluster, model_name, "manual",
            data_type, data_path, manual_settings)
        results.append(("manual", manual_time, manual_mem))

        skl_cluster, skl_time, skl_mem = run_with_profile(train_func, x
            , skl_settings)
        evaluate_result(x, y, skl_cluster, model_name, "sklearn",
            data_type, data_path, skl_settings)
        results.append(("sklearn", skl_time, skl_mem))
    else:
        if model_name == "dbscan" and len(base_settings) <= 2:
            base_settings = tuple(base_settings) + ("sklearn",)
        cluster, t_cost, mem_cost = run_with_profile(train_func, x,
            base_settings)
        run_name = getattr(cluster, "implementation", "single_run")
        evaluate_result(x, y, cluster, model_name, run_name, data_type,
            data_path, base_settings)
```

```
        results.append((run_name, t_cost, mem_cost))

    print("\n时间/内存统计:")
    for name, t_cost, mem_cost in results:
        print("  %s: time=%.4fs, peak_mem=%.2fMB" % (name, t_cost,
           mem_cost))


def run_sweep(model_name, x, y, param_grid, data_type, data_path):
    print("开始参数扫描, 共%d组参数。" % len(param_grid))
    for settings in param_grid:
        print("\n参数: %s" % (settings,))
        run_model(model_name, x, y, settings=settings, compare=False,
           data_type=data_type, data_path=data_path)


def sample_if_needed(x, y, max_samples, random_state=42):
    if max_samples is None or x.shape[0] <= max_samples:
        return x, y
    rng = np.random.default_rng(random_state)
    idx = rng.choice(x.shape[0], size=max_samples, replace=False)
    x_s = x[idx]
    y_s = y[idx] if y is not None else None
    print("数据量较大, 已随机采样到%d条以便完成层次聚类实验。" %
       max_samples)
    return x_s, y_s


def parse_args():
    parser = argparse.ArgumentParser(description="Clustering Lab Runner
       ")
    parser.add_argument("-d", "--data", required=True, help="数据文件或
       目录路径")
    parser.add_argument("-t", "--type", required=True, choices=["dot",
       "timeseries", "uci"], help="数据类型")
    parser.add_argument("-m", "--model", default="all", choices=["
       kmeans", "linkage", "dbscan", "all"], help="聚类算法")
    parser.add_argument("-s", "--settings", default=None, help="算法参
       数, 如 '(5, \"random\")'")
    parser.add_argument("--compare", action="store_true", default=True,
        help="对kmeans/linkage执行手写vs sklearn对比")
```

```
    parser.add_argument("--no-compare", dest="compare", action="
        store_false", help="关闭手写vs sklearn对比")
    parser.add_argument("--sweep", action="store_true", help="是否执行
        参数扫描")
    parser.add_argument("--max-linkage-samples", type=int, default
        =1200, help="linkage默认最大样本数")
    return parser.parse_args()


def main():
    args = parse_args()
    settings = ast.literal_eval(args.settings) if args.settings else
        None
    ensure_result_dir()

    x_train, y_train = load_data(args.data, args.type)

    model_list = ["kmeans", "linkage", "dbscan"] if args.model == "all"
        else [args.model]
    for model_name in model_list:
        x_use, y_use = x_train, y_train
        if model_name == "linkage":
            x_use, y_use = sample_if_needed(x_train, y_train, args.
                max_linkage_samples)

        print("\n" + "=" * 70)
        print("当前算法: %s" % model_name)
        print("=" * 70)

        if args.sweep:
            if model_name == "kmeans":
                grid = [(3, "random", "manual"), (5, "random", "manual"
                    ), (8, "random", "manual")]
            elif model_name == "linkage":
                grid = [(3, "single", "manual"), (5, "single", "manual"
                    ), (8, "single", "manual")]
            else:
                grid = [(0.4, 5, "sklearn"), (0.8, 10, "sklearn"),
                    (1.2, 15, "sklearn")]
            run_sweep(model_name, x_use, y_use, grid, data_type=args.
                type, data_path=args.data)
```

```
        else:
            run_model(
                model_name,
                x_use,
                y_use,
                settings=settings,
                compare=args.compare,
                data_type=args.type,
                data_path=args.data,
            )


if __name__ == "__main__":
    main()
```

A.2 kmeans.py

```
    # -*- coding: utf-8 -*-
from types import SimpleNamespace
import numpy as np
from sklearn.cluster import KMeans
import time


ModelName = "KMeans聚类算法"


def _manual_kmeans(data, n_clusters, init="random", max_iter=300, tol=1
   e-4, random_state=42):
    rng = np.random.default_rng(random_state)
    n_samples = data.shape[0]
    if n_clusters <= 0 or n_clusters > n_samples:
        raise ValueError("n_clusters must be in [1, n_samples].")

    if init == "random":
        centers = data[rng.choice(n_samples, size=n_clusters, replace=
           False)].copy()
    else:
        raise ValueError("Only 'random' initialization is supported in
           manual KMeans.")
```

```
    labels = np.zeros(n_samples, dtype=np.int32)
    for it in range(max_iter):
        distances = np.linalg.norm(data[:, None, :] - centers[None, :,
            :], axis=2)
        new_labels = np.argmin(distances, axis=1)

        new_centers = centers.copy()
        for i in range(n_clusters):
            members = data[new_labels == i]
            if members.size == 0:
                new_centers[i] = data[rng.integers(0, n_samples)]
            else:
                new_centers[i] = members.mean(axis=0)

        center_shift = np.linalg.norm(new_centers - centers)
        centers = new_centers
        labels = new_labels
        if center_shift <= tol:
            break

    inertia = float(np.sum((data - centers[labels]) ** 2))
    return SimpleNamespace(
        labels_=labels,
        cluster_centers_=centers,
        n_iter_=it + 1,
        inertia_=inertia,
        implementation="manual",
    )


def train(data, *args):
    print("开始%s过程..." % ModelName)
    startT = time.time()

    n_clusters = args[0] if len(args) > 0 else 5
    init = args[1] if len(args) > 1 else "random"
    implementation = args[2].lower() if len(args) > 2 else "manual"
    max_iter = int(args[3]) if len(args) > 3 else 300
    tol = float(args[4]) if len(args) > 4 else 1e-4
    random_state = int(args[5]) if len(args) > 5 else 42
```

```
    if implementation == "manual":
        cluster = _manual_kmeans(
            data,
            n_clusters=n_clusters,
            init=init,
            max_iter=max_iter,
            tol=tol,
            random_state=random_state,
        )
    elif implementation == "sklearn":
        cluster = KMeans(
            n_clusters=n_clusters,
            init=init,
            max_iter=max_iter,
            tol=tol,
            n_init=10,
            random_state=random_state,
        ).fit(data)
        cluster.implementation = "sklearn"
    else:
        raise ValueError("implementation must be one of {'manual', '
           sklearn'}")

    endT = time.time()
    print("%s过程结束。处理了%d个数据点，耗时%f秒。" % (ModelName, data
       .shape[0], (endT - startT)))
    return cluster
```

A.3 linkage.py

```
    # -*- coding: utf-8 -*-
from types import SimpleNamespace
import numpy as np
from sklearn.cluster import AgglomerativeClustering
import time


ModelName = "凝聚法层次聚类算法"
```

```python
def _slink_edges(data):
    n_samples = data.shape[0]
    pi = np.arange(n_samples, dtype=np.int32)
    lam = np.full(n_samples, np.inf, dtype=np.float64)

    for i in range(1, n_samples):
        m = np.linalg.norm(data[i] - data[:i], axis=1)
        pi[i] = i
        lam[i] = np.inf
        for j in range(i):
            pj = pi[j]
            if lam[j] >= m[j]:
                m[pj] = min(m[pj], lam[j])
                lam[j] = m[j]
                pi[j] = i
            else:
                m[pj] = min(m[pj], m[j])
        for j in range(i):
            if lam[j] >= lam[pi[j]]:
                pi[j] = i

    edges = [(i, int(pi[i]), float(lam[i])) for i in range(n_samples)
       if pi[i] != i]
    return edges


def _manual_slink(data, n_clusters):
    n_samples = data.shape[0]
    if n_clusters <= 0 or n_clusters > n_samples:
        raise ValueError("n_clusters must be in [1, n_samples].")

    edges = _slink_edges(data)
    edges.sort(key=lambda x: x[2])

    parent = np.arange(n_samples, dtype=np.int32)
    rank = np.zeros(n_samples, dtype=np.int8)

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
```

```
            x = parent[x]
        return x

    def union(a, b):
        ra, rb = find(a), find(b)
        if ra == rb:
            return
        if rank[ra] < rank[rb]:
            parent[ra] = rb
        elif rank[ra] > rank[rb]:
            parent[rb] = ra
        else:
            parent[rb] = ra
            rank[ra] += 1

    # Keep n_clusters components by adding only (n - n_clusters)
       shortest edges.
    for a, b, _dist in edges[: max(0, n_samples - n_clusters)]:
        union(a, b)

    roots = np.array([find(i) for i in range(n_samples)], dtype=np.
       int32)
    unique_roots, labels = np.unique(roots, return_inverse=True)

    return SimpleNamespace(
        labels_=labels,
        n_clusters_=unique_roots.size,
        implementation="manual",
        linkage="single",
    )


def train(data, *args):
    print("开始%s过程..." % ModelName)
    startT = time.time()

    n_clusters = args[0] if len(args) > 0 else 5
    linkage_name = args[1] if len(args) > 1 else "single"
    implementation = args[2].lower() if len(args) > 2 else "manual"

    if implementation == "manual":
```

```
        if linkage_name != "single":
            raise ValueError("Manual implementation only supports '
               single' linkage (SLINK).")
        cluster = _manual_slink(data, n_clusters=n_clusters)
    elif implementation == "sklearn":
        cluster = AgglomerativeClustering(n_clusters=n_clusters,
           linkage=linkage_name).fit(data)
        cluster.implementation = "sklearn"
    else:
        raise ValueError("implementation must be one of {'manual', '
           sklearn'}")

    endT = time.time()
    print("%s过程结束。处理了%d个数据点，耗时%f秒。" % (ModelName, data
       .shape[0], (endT - startT)))
    return cluster
```

A.4 features.py

```
    # -*- coding: utf-8 -*-
import numpy as np


def dotFeature(x, _settings=None):
    """Dots are already 2D features; return as float array."""
    return np.asarray(x, dtype=np.float64), {"type": "identity"}


def timeSeriesFeature(x, _settings=None):
    """Extract compact statistical features from each time series."""
    x = np.asarray(x, dtype=np.float64)

    mean = x.mean(axis=1)
    std = x.std(axis=1)
    min_v = x.min(axis=1)
    max_v = x.max(axis=1)

    dx = np.diff(x, axis=1)
    slope_mean = dx.mean(axis=1)
    slope_std = dx.std(axis=1)
```

```
    feats = np.stack([mean, std, min_v, max_v, slope_mean, slope_std],
        axis=1)
    return feats, {"type": "stats6"}
```

A.5 运行命令

```
# 二维点数据：同时比较三种算法
python clustering.py -d dots/clusterData1.10k.dat -t dot -m all

# 时序数据：比较三种算法
python clustering.py -d timeSeries/Plane_TRAIN.txt -t timeseries -m all

# UCI 数据：比较三种算法
python clustering.py -d wine+quality -t uci -m all

# 参数扫描
python clustering.py -d wine+quality -t uci -m all --sweep
```